

Lab05_ME315A

Author

Nathan Recco Neves - 277213

Lab 05

Pacotes utilizados

Tidyverse

```
if (!require("tidyverse", character.only = TRUE)) {  
  install.packages("tidyverse")  
  library("tidyverse")  
}
```

—

Carregando pacotes exigidos: tidyverse

— Attaching core tidyverse packages — tidyverse
2.0.0 —

✓ dplyr	1.2.1	✓ readr	2.2.0
✓ forcats	1.0.1	✓ stringr	1.6.0
✓ ggplot2	4.0.3	✓ tibble	3.3.1
✓ lubridate	1.9.5	✓ tidyr	1.3.2
✓ purrr	1.2.2		

— Conflicts —

tidyverse_conflicts() —

✖ dplyr::filter() masks stats::filter()

✖ dplyr::lag() masks stats::lag()

i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

ggcal

```
library(ggcal)
```

—

Leaflet

```
if (!require("leaflet", character.only = TRUE)) {  
  install.packages("leaflet")  
  library("leaflet")  
}
```

—

Carregando pacotes exigidos: leaflet

geosphere

```
if (!require("geosphere", character.only = TRUE)) {  
  install.packages("geosphere")  
  library("geosphere")  
}
```

—

Carregando pacotes exigidos: geosphere

```
analisa_aeronave <- function(tail_number, arquivo) {  
  
  # 1. Leitura do arquivo em blocos (chunks)  
  voos_aeronave <- read_csv_chunked(  
    file = arquivo,  
    callback = DataFrameCallback$new(function(chunk, pos) {  
      chunk %>% filter(TAIL_NUMBER == tail_number)  
    }),  
    chunk_size = 50000,  
    show_col_types = FALSE  
  )  
  
  if (nrow(voos_aeronave) == 0) {  
    stop(paste("Nenhum voo encontrado para a aeronave:", tail_number))  
  }  
  
  # 2. Tratamento do horário de partida sem gerar avisos de falha  
  voos_aeronave <- voos_aeronave %>%  
    mutate(  
      data_voo = make_date(year = YEAR, month = MONTH, day = DAY),  
      # Ajusta horários do tipo "2400" para "2359" para não quebrar a data  
      DEPARTURE_TIME_CLEAN = if_else(DEPARTURE_TIME == 2400, 2359,  
        as.numeric(DEPARTURE_TIME)),  
      hora_partida = sprintf("%04d", as.integer(DEPARTURE_TIME_CLEAN)),  
      horario_partida = suppressWarnings(  
        ymd_hm(paste(data_voo, sprintf("%s:%s", substr(hora_partida, 1, 2),  
          substr(hora_partida, 3, 4))))  
      )  
    ) %>%  
    arrange(horario_partida)  
  
  # 3. Carregamento dos dados de aeroportos  
  caminho_airports <- file.path(dirname(arquivo), "airports.csv")  
  
  if (!file.exists(caminho_airports)) {  
    stop("O arquivo airports.csv não foi encontrado no mesmo diretório.")  
  }  
}
```

```

airports <- read_csv(caminho_airports, show_col_types = FALSE) %>%
  select(IATA_CODE, LATITUDE, LONGITUDE)

# 4. Cruzamento de coordenadas
voos_mapa <- voos_aeronave %>%
  left_join(airports, by = c("ORIGIN_AIRPORT" = "IATA_CODE")) %>%
  rename(latitude_origem = LATITUDE, longitude_origem = LONGITUDE) %>%
  left_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE")) %>%
  rename(latitude_destino = LATITUDE, longitude_destino = LONGITUDE)

# 5. Métricas de distância e velocidade
voos_mapa <- voos_mapa %>%
  mutate(
    distancia_km = distHaversine(
      cbind(longitude_origem, latitude_origem),
      cbind(longitude_destino, latitude_destino)
    ) / 1000,
    velocidade_media_kmh = if_else(!is.na(AIR_TIME) & AIR_TIME > 0,
      distancia_km / (AIR_TIME / 60), NA_real_),
    numero_voo = row_number()
  )

# 6. Mapeamento de marcadores (CORRIGIDO: longitude_destino mapeada
  corretamente)
aeroportos_utilizados <- bind_rows(
  voos_mapa %>% select(aeroporto = ORIGIN_AIRPORT, latitude =
    latitude_origem, longitude = longitude_origem),
  voos_mapa %>% select(aeroporto = DESTINATION_AIRPORT, latitude =
    latitude_destino, longitude = longitude_destino)
) %>%
  distinct(aeroporto, .keep_all = TRUE) %>%
  filter(!is.na(latitude) & !is.na(longitude))

# 7. Construção do mapa
mapa <- leaflet() %>%
  addTiles() %>%
  addCircleMarkers(
    data = aeroportos_utilizados,
    lng = ~longitude,
    lat = ~latitude,
    radius = 5,
    color = "#D32F2F",
    fillColor = "#E53935",
    fillOpacity = 0.9,
    popup = ~paste0("<b>Aeroporto:</b> ", aeroporto)
  )

# Adiciona linhas para cada trecho percorrido
for (i in seq_len(nrow(voos_mapa))) {
  voo <- voos_mapa[i, ]

```

```

if (!is.na(voo$latitude_origem) && !is.na(voo$latitude_destino)) {
  espessura <- ifelse(is.na(voo$velocidade_media_kmh), 2, max(1,
    min(voo$velocidade_media_kmh / 100, 8)))

  popup_info <- paste0(
    "<b>Trecho #</b>", voo$numero_voo, "<br>",
    "<b>Rota:</b> ", voo$ORIGIN_AIRPORT, " &rarr; ",
    voo$DESTINATION_AIRPORT, "<br>",
    "<b>Data:</b> ", voo$data_voo, "<br>",
    "<b>Distância:</b> ", round(voo$distancia_km, 1), " km<br>",
    "<b>Velocidade Média:</b> ", round(voo$velocidade_media_kmh, 1), "
    km/h"
  )

  mapa <- mapa %>%
    addPolylines(
      lng = c(voo$longitude_origem, voo$longitude_destino),
      lat = c(voo$latitude_origem, voo$latitude_destino),
      color = "#1E88E5",
      weight = espessura,
      opacity = 0.6,
      popup = popup_info
    )
}
}

mapa <- mapa %>%
  addControl(
    html = paste0("<b>Aeronave:</b> ", tail_number, "<br><small>Espessura
    = Velocidade Média</small>"),
    position = "topright"
  )

return(list(tabela = voos_mapa, mapa = mapa))
}

```

□

Verificando que o arquivo file.csv existe

Vendo se o arquivo está no mesmo diretório

```
file.exists("flights.csv")
```

□

```
[1] TRUE
```

Tabela

Aplicando a função no arquivo

```

resultado <- analisa_aeronave(
  tail_number = "N431WN",
  arquivo = "flights.csv"
)

```

□

Mapa

Apresentando o mapa

```
resultado$tabela
```

□

```

# A tibble: 1,938 × 42
  YEAR MONTH DAY DAY_OF_WEEK AIRLINE FLIGHT_NUMBER TAIL_NUMBER
  <dbl> <dbl> <dbl>         <dbl> <chr>          <dbl> <chr>
1  2015     1     1             4 WN            1762 N431WN
2  2015     1     1             4 WN            3564 N431WN
3  2015     1     1             4 WN            3564 N431WN
4  2015     1     1             4 WN            3564 N431WN
5  2015     1     2             5 WN            1793 N431WN
6  2015     1     2             5 WN            1793 N431WN
7  2015     1     2             5 WN            1793 N431WN
8  2015     1     2             5 WN            1793 N431WN
9  2015     1     2             5 WN            1793 N431WN
10 2015     1     2             5 WN            3144 N431WN
# i 1,928 more rows
# i 35 more variables: ORIGIN_AIRPORT <chr>, DESTINATION_AIRPORT <chr>,
#   SCHEDULED_DEPARTURE <chr>, DEPARTURE_TIME <chr>, DEPARTURE_DELAY <dbl>,
#   TAXI_OUT <dbl>, WHEELS_OFF <chr>, SCHEDULED_TIME <dbl>, ELAPSED_TIME
#   <dbl>,
#   AIR_TIME <dbl>, DISTANCE <dbl>, WHEELS_ON <chr>, TAXI_IN <dbl>,
#   SCHEDULED_ARRIVAL <chr>, ARRIVAL_TIME <chr>, ARRIVAL_DELAY <dbl>,
#   DIVERTED <dbl>, CANCELLED <dbl>, CANCELLATION_REASON <chr>, ...

# Apresentando o mapa

```

```
resultado$mapa
```

□

Horário

```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

—

```
[1] "2026-09-16 17:55:27 -03"
```